Novosibirsk National Research State University

Database Browser

Topic:
Development of an Integrated Development Environment for Designing and Analyzing Relational Databases

Completed by:
24215 Pisarev Artem Sergeevich

24215 Vorona Mikhail Evgenievich

24215 Maltsev Kirill Konstantinovich

Novosibirsk - 2026

Table of Contents

Introduction....4
Domain Glossary....5
Functional Requirements....6
Non-Functional Requirements....8
Technology Stack....9
Algorithms....11
Sketches....15
Precedents....19

Introduction

Our goal is to create an intuitive, high-performance Integrated Development Environment (IDE) that empowers developers and data analysts to seamlessly design, manage, and optimize relational databases. By prioritizing SQLite support while maintaining extensibility for popular RDBMSs, the tool aims to bridge the gap between complex database administration and user-friendly visual interaction.

Domain Glossary

RDBMS (Relational Database Management System) - A DBMS based on the relational data model (SQLite, PostgreSQL, MySQL, etc.).

SQLite - A lightweight relational DBMS that stores the entire database in a single file on disk.

Diagram - A graphical representation of information, data, processes, or relationships between objects using geometric shapes, lines, symbols, or graphs.

Overview Window - A user interface (UI) element that displays a reduced, summarized, or full version of a large object (document, image, map, dataset, or workspace) as a separate panel or window.

DDL (Data Definition Language) - A subset of SQL used to create and modify the database structure (CREATE, ALTER, DROP commands).

Database Schema - The logical structure of the database: tables, columns, data types, constraints, indexes, and views.

Result Cardinality - The number of rows returned as a result of executing an SQL query.

Query Execution Plan - The sequence of steps that the DBMS plans to execute to obtain the query result. Used to identify "bottlenecks".

Functional Requirements

Connection Management FR-1:

- FR-1.1 The system must allow creating, editing, and deleting connections to SQLite database files.

- FR-1.2 The system must verify the availability and integrity of the DB file upon connection.

DataBase Schema Management (DDL) FR-2:

- FR-2.1 The system must provide a visual or text-based interface for creating, modifying, and deleting tables.

- FR-2.2 The system must support constraint management (PRIMARY KEY, FOREIGN KEY, UNIQUE, CHECK, NOT NULL).

- FR-2.3 The system must allow creating, modifying, and deleting indexes and views.

- FR-2.4 The system must provide the ability to export the current DB schema as a single SQL script (DDL).

Query Execution and Data Visualization FR-3:

- FR-3.1 The system must allow entering and executing arbitrary SQL queries (SELECT, INSERT, UPDATE, DELETE, etc.).

- FR-3.2 The system must display the results of SELECT queries as a table (Data Grid).

- FR-3.3 For high result cardinality, the system must load and display data in chunks without loading the entire volume into RAM.

- FR-3.4 The system must display general query statistics: execution time, number of affected rows, and total number of rows in the result (asynchronously).

Query Plan Analysis and Visualization FR-4:

- FR-4.1 The system must extract the query execution plan (e.g., using EXPLAIN QUERY PLAN in SQLite).

- FR-4.2 The system must visualize the query plan as a tree or graph.

- FR-4.3 The system must highlight potentially suboptimal operations (e.g., Full Table Scan).

Architecture Scalability FR-5:

- FR-5.1 The architecture must allow adding support for other DBMSs (PostgreSQL, MySQL) in the future without rewriting the system core (using Strategy/Plugin patterns).

Platform and Technology Stack FR-6:

- FR-6.1 The system must be implemented as a desktop application for the Windows operating system

- FR-6.2 The technology stack must be based on Python/PyQt

Non-Functional Requirements

Reliability NFR-1:

- NFR-1.1The system must not crash when invalid SQL syntax is entered or when working with corrupted DB files. Errors must be caught and displayed to the user.

Usability NFR-2:

- NFR-2.1 The interface must support SQL syntax highlighting, autocomplete, and dark/light themes.

Resource Utilization(CPU) NFR-3:

- NFR-3.1 The system must not consume more than 5% of CPU resources in an idle state.

- NFR-3.2 During intensive background tasks(e. g., executing heavy queries), the application must limit CPU usage to a maximum of 30% to maintain host OS responsiveness.

Hosting and Deployment NFR-4:

- NFR-4.1 The system must be hosted and executed locally on the end-user's Windows machine without requiring a dedicated remote backend server or cloud infrastructure.

- NFR-4.2 The application must be packaged as a standalone local executable (e. g., .exe) that runs entirely within the local user-space environment

Technology Stack

The technology stack is selected to support a local Windows desktop application for database schema management, SQL query execution, and query plan analysis.

Programming Language: Python

Python is chosen because its readable syntax and available libraries help a small team develop and modify the application within the project schedule. Its standard library provides tools for working with files and SQLite databases, reducing the number of additional dependencies. Python will handle application logic and result processing, while SQLite will execute SQL queries.

User Interface Framework: PyQt

PyQt is chosen because it provides access to the Qt framework directly from Python. It offers the tools needed to build the SQL editor, display database schemas and query results, and visualize execution plans.

Qt supports custom data models and loading additional data on demand, which suits the requirement to display large query results in chunks. Its mechanisms for background work and communication between objects can also be used to keep the interface responsive during long database operations. Using one framework for these tasks simplifies integration.

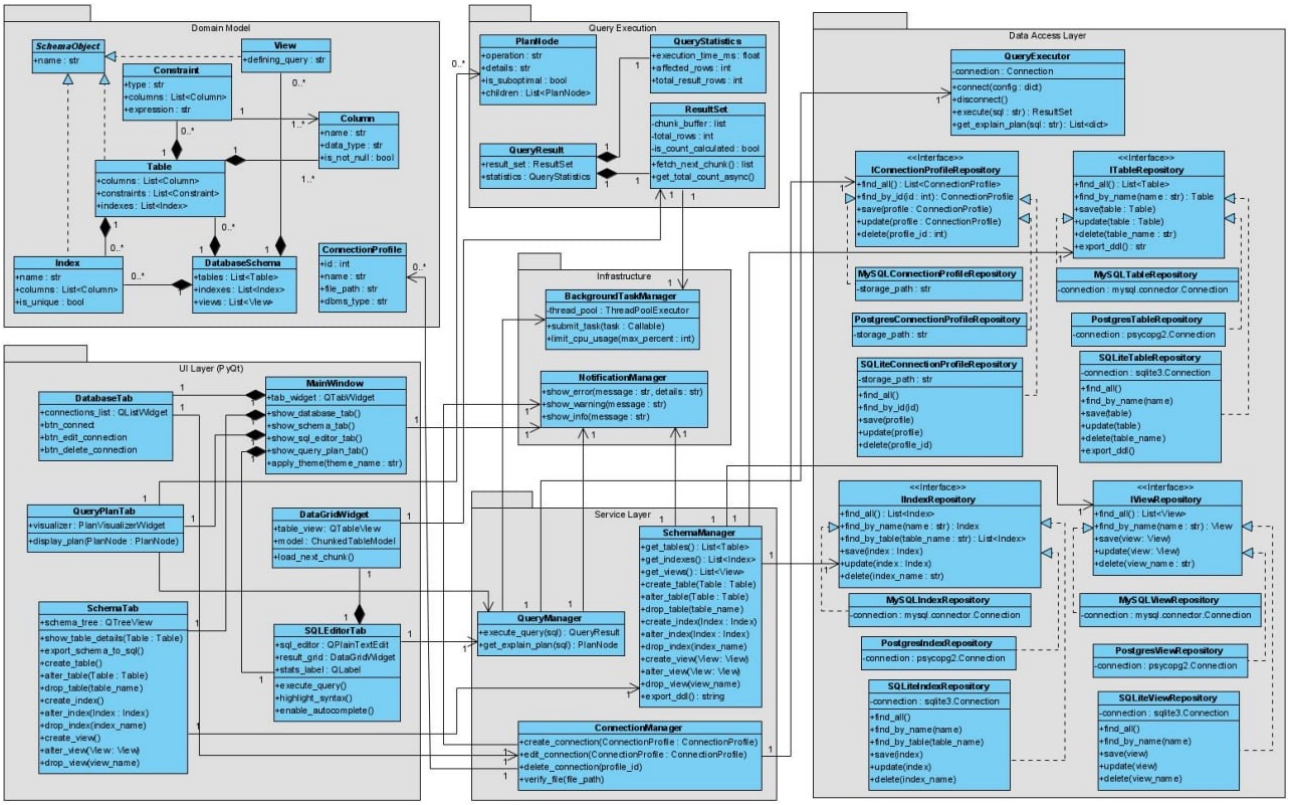
Database Access: SQLite and sqlite3

SQLite is chosen as the first supported DBMS because it works with local database files and does not require a separate database server. This matches the requirement to run the application locally and makes it easier to prepare databases for development, testing, and demonstration.

The application will access SQLite through Python's standard sqlite3 module. It provides SQL execution, transaction control, and cursor-based result retrieval. Direct SQL access is suitable for a database browser because users need to execute their own queries and modify database structures.

Application Packaging: PyInstaller

PyInstaller is proposed for distributing the application on Windows. It can bundle the Python interpreter, application code, and required libraries into a package with an executable file. Users will therefore be able to launch the application without installing Python and its dependencies separately. The release package will be built and tested on Windows.


Domain Model
SchemaObject
+name : str
View
+defining_query : str
Constraint
+type : str
+columns : List<Column>
+expression : str
Column
+name : str
+data_type : str
+is_not_null : bool
Table
+columns : List<Column>
+constraints : List<Constraint>
+indexes : List<Index>
Index
+name : str
+columns : List<Column>
+is_unique : bool
DatabaseSchema
+tables : List<Table>
+indexes : List<Index>
+views : List<View>
ConnectionProfile
+id : int
+name : str
+file_path : str
+dbms_type : str
Query Execution
PlanNode
+operation : str
+details : str
+is_suboptimal : bool
+children : List<PlanNode>
QueryStatistics
+execution_time_ms : float
+affected_rows : int
+total_result_rows : int
ResultSet
-chunk_buffer : list
-total_rows : int
-is_count_calculated : bool
+fetch_next_chunk() : list
+get_total_count_async()
QueryResult
+result_set : ResultSet
+statistics : QueryStatistics
Data Access Layer
QueryExecutor
-connection : Connection
+connect(config : dict)
+disconnect()
+execute(sql : str) : ResultSet
+get_explain_plan(sql : str) : List<dict>
<<Interface>>
IConnectionProfileRepository
+find_all() : List<ConnectionProfile>
+find_by_id(id : int) : ConnectionProfile
+save(profile : ConnectionProfile)
+update(profile : ConnectionProfile)
+delete(profile_id : int)
<<Interface>>
ITableRepository
+find_all() : List<Table>
+find_by_name(name : str) : Table
+save(table : Table)
+update(table : Table)
+delete(table_name : str)
+export_ddl() : str
MySQLConnectionProfileRepository
-storage_path : str
MySQLTableRepository
-connection : mysql.connector.Connection
PostgresConnectionProfileRepository
-storage_path : str
PostgresTableRepository
-connection : psycopg2.Connection
SQLiteConnectionProfileRepository
-storage_path : str
+find_all()
+find_by_id(id)
+save(profile)
+update(profile)
+delete(profile_id)
SQLiteTableRepository
-connection : sqlite3.Connection
+find_all()
+find_by_name(name)
+save(table)
+update(table)
+delete(table_name)
+export_ddl()
Infrastructure
BackgroundTaskManager
-thread_pool : ThreadPoolExecutor
+submit_task(task : Callable)
+limit_cpu_usage(max_percent : int)
NotificationManager
+show_error(message : str, details : str)
+show_warning(message : str)
+show_info(message : str)
UI Layer (PyQt)
DatabaseTab
+connections_list : QListWidget
+btn_connect
+btn_edit_connection
+btn_delete_connection
MainWindow
+tab_widget : QTabWidget
+show_database_tab()
+show_schema_tab()
+show_sql_editor_tab()
+show_query_plan_tab()
+apply_theme(theme_name : str)
QueryPlanTab
+visualizer : PlanVisualizerWidget
+display_plan(PlanNode : PlanNode)
DataGridWidget
+table_view : QTableView
+model : ChunkedTableModel
+load_next_chunk()
SchemaTab
+schema_tree : QTreeView
+show_table_details(Table : Table)
+export_schema_to_sql()
+create_table()
+alter_table(Table : Table)
+drop_table(table_name)
+create_index()
+alter_index(Index : Index)
+drop_index(index_name)
+create_view()
+alter_view(View: View)
+drop_view(view_name)
SQLEditorTab
+sql_editor : QPlainTextEdit
+result_grid : DataGridWidget
+stats_label : QLabel
+execute_query()
+highlight_syntax()
+enable_autocomplete()
Service Layer
QueryManager
+execute_query(sql) : QueryResult
+get_explain_plan(sql) : PlanNode
SchemaManager
+get_tables() : List<Table>
+get_indexes() : List<Index>
+get_views() : List<View>
+create_table(Table : Table)
+alter_table(Table : Table)
+drop_table(table_name)
+create_index(Index : Index)
+alter_index(Index : Index)
+drop_index(index_name)
+create_view(View: View)
+alter_view(View: View)
+drop_view(view_name)
+export_ddl() : string
ConnectionManager
+create_connection(ConnectionProfile : ConnectionProfile)
+edit_connection(ConnectionProfile : ConnectionProfile)
+delete_connection(profile_id)
+verify_file(file_path)
<<Interface>>
IIndexRepository
+find_all() : List<Index>
+find_by_name(name : str) : Index
+find_by_table(table_name : str) : List<Index>
+save(index : Index)
+update(index : Index)
+delete(index_name : str)
<<Interface>>
IViewRepository
+find_all() : List<View>
+find_by_name(name : str) : View
+save(view: View)
+update(view: View)
+delete(view_name : str)
MySQLIndexRepository
-connection : mysql.connector.Connection
MySQLViewRepository
-connection : mysql.connector.Connection
PostgresIndexRepository
-connection : psycopg2.Connection
PostgresViewRepository
-connection : psycopg2.Connection
SQLiteIndexRepository
-connection : sqlite3.Connection
+find_all()
+find_by_name(name)
+find_by_table(table_name)
+save(index)
+update(index)
+delete(index_name)
SQLiteViewRepository
-connection : sqlite3.Connection
+find_all()
+find_by_name(name)
+save(view)
+update(view)
+delete(view_name)

Algorithms

These two algorithms use the classes and methods in the Database Browser class diagram.

Loading query results in portions

Purpose

Show query results in small portions so that the user does not have to wait until all rows are loaded into the application.

Main classes and methods

QueryManager.execute_query(sql): Coordinates query execution and prepares
QueryResult QueryExecutor.execute(sql): Executes SQL and returns
ResultSet ResultSet.fetch_next_chunk(): Reads the next portion of
rows DataGridWidget.load_next_chunk(): Requests more rows for the result table

QueryResult contains the result set and QueryStatistics. Background work uses
BackgroundTaskManager.submit_task(task).

Input and output

Input: a SQL statement, an active connection, and an internal chunk size greater than zero.

Output: portions of result rows displayed in the table, together with the available query statistics.

Steps

1. The user runs a statement in the SQL editor. The editor calls QueryManager.execute_query(sql).
2. The manager arranges background execution and calls QueryExecutor.execute(sql). The executor runs the statement once and returns a ResultSet that keeps access to its cursor.
3. The manager prepares QueryResult with the result set and statistics. The UI receives it and prepares the result table.
4. The grid requests rows through load_next_chunk(). The result set reads up to the configured chunk size using fetch_next_chunk(). In SQLite, this can use cursor.fetchmany().
5. The UI adds the returned rows to its table model. When the user requests more data, the application repeats step 4 using the same cursor.
6. When an empty portion is returned, all rows have been read. The application records the final row count and releases the cursor.
7. If the statement has no result columns, the application shows its available execution statistics without loading table rows.
8. If reading fails or the result is no longer needed, the background worker releases the cursor. Rows from an old query must not be added to a new result table.

Errors and limitations

SQL errors, database locks, and connection errors are shown through NotificationManager. If reading stops after some rows were displayed, the result is marked as incomplete.

Query execution and fetching run outside the UI thread. Widget updates run in the UI thread. The owning worker handles the connection and cursor. It processes fetch requests one at a time.

The number of displayed rows may differ from the total result count. The existing ResultSet.get_total_count_async() can request a count in the background. Until an exact count is available, is_count_calculated remains false. Counting must not repeat statements that change data.

The query is not run again for each portion. Chunking reduces the amount loaded at once, but memory still grows if the table keeps every loaded row. Limits and cache rules require a separate decision. Some queries may also take time before returning their first rows.

Checks

Check an empty result, one portion, several portions, a final portion smaller than the chunk size, an exact multiple of the chunk size, a statement without result columns, and an error while reading. Rows must not be skipped or duplicated.

Building a query-plan tree

Purpose

Turn the rows returned by SQLite into a tree of PlanNode objects for display. SQLite creates the execution plan; the application builds its visual structure.

Main classes and methods

QueryExecutor.get_explain_plan(sql): Returns plan rows as
List QueryManager.get_explain_plan(sql): Converts those rows into a root PlanNode PlanNode: Stores operation, details, is_suboptimal, and children QueryPlanTab.display_plan(PlanNode):

Displays the tree

Tree building is part of the manager's existing method. A separate builder class is not required.

Input and output

Input: a SQL statement and an active database connection.

Output: one root PlanNode containing the plan nodes in its children hierarchy.

Steps

1. The query manager arranges background work and calls QueryExecutor.get_explain_plan(sql).
2. The executor runs EXPLAIN QUERY PLAN for the statement. It returns plan rows without executing the original statement.
3. The manager checks the returned format. Each supported row contains a node ID, a parent ID, an auxiliary value, and a description.
4. For each row, create a PlanNode. Store the full description in details and set operation to a recognized operation label, or a general label if it is unknown. Keep the nodes in a temporary dictionary by ID, rejecting duplicate IDs before insertion.
5. Check the parent links. Identify top-level nodes using the supported SQLite format. Reject invalid self-links, missing non-root parents, and cycles.
6. Create an extra root named QUERY PLAN. It allows several top-level nodes to be returned as one tree.
7. Go through the rows again. Add each node to its parent's children, or to the extra root if it is a top-level node. Keep the original order. Parents may appear after their children because all nodes already exist.
8. Return the root to the UI and display it through QueryPlanTab.display_plan(root). If no plan rows were returned, show an empty root with “No plan information.”

IDs are temporary data and do not require new fields in PlanNode.

Errors and limitations

Report SQL, connection, and plan-format errors through NotificationManager. Do not display a tree with invalid links.

SQLite's plan format can change between versions, so the conversion must be checked for the versions the application supports. Unknown descriptions are preserved.

A table scan alone does not prove that a query is inefficient. Set is_suboptimal only according to separately agreed warning rules. Until such rules exist, false means “not flagged,” not “proven optimal.”

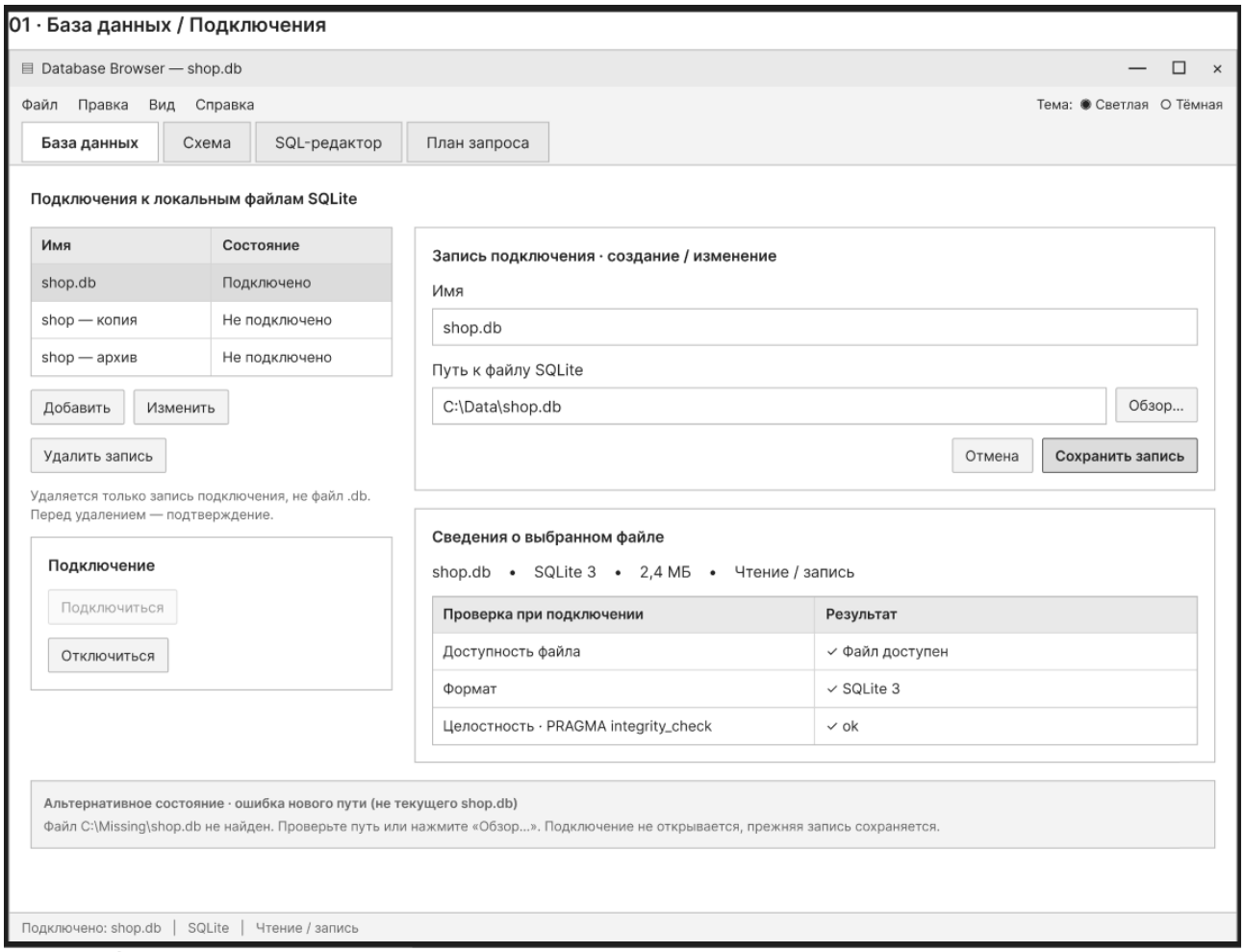
The plan shows how SQLite intends to process the query. It does not provide actual execution time or actual row counts. Building the tree takes linear time and memory, excluding text and display layout.

Checks

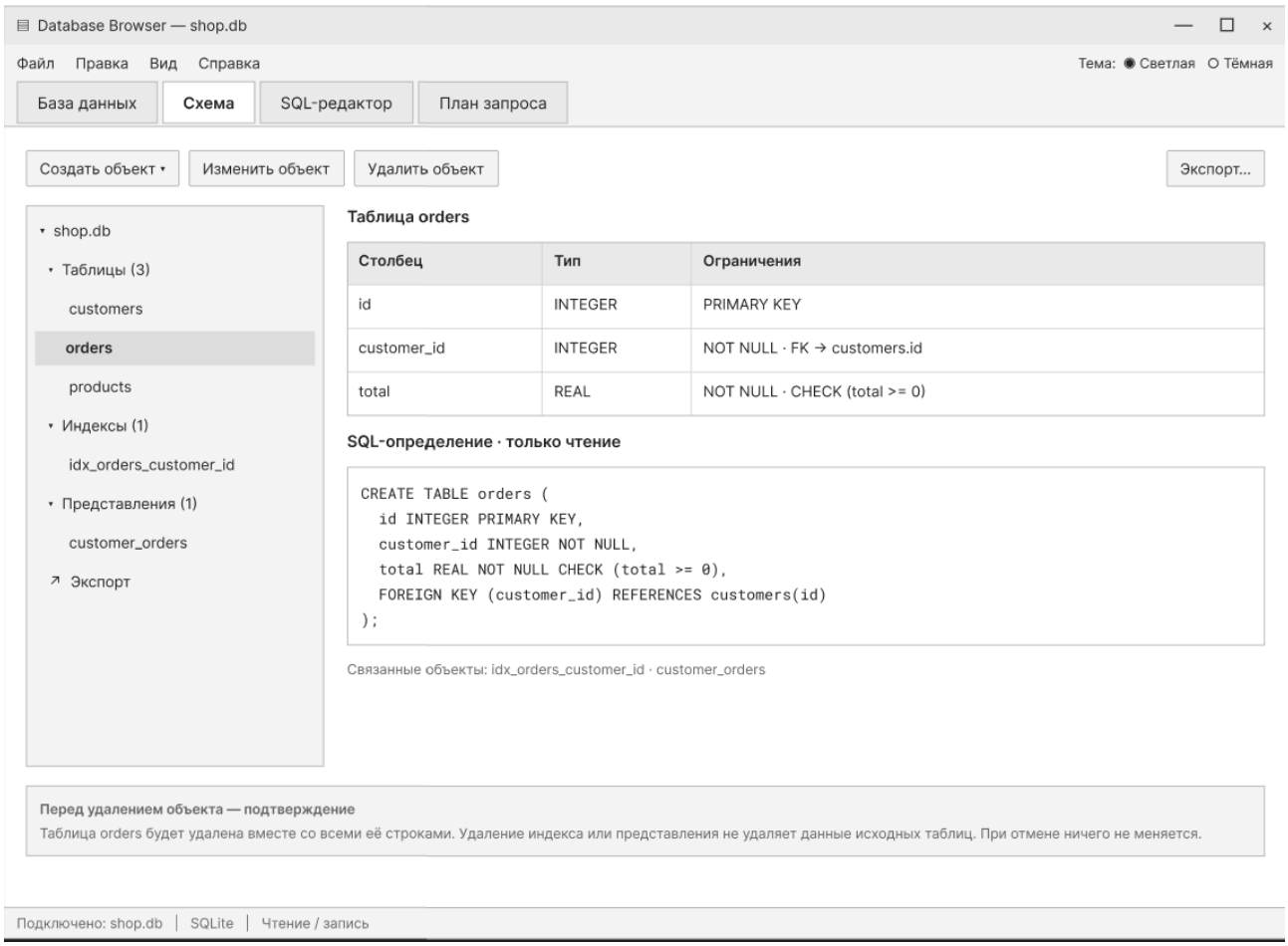
Check one node, several top-level nodes, nested nodes, a child appearing before its parent, unknown descriptions, duplicate IDs, invalid parents, cycles, and an empty plan.

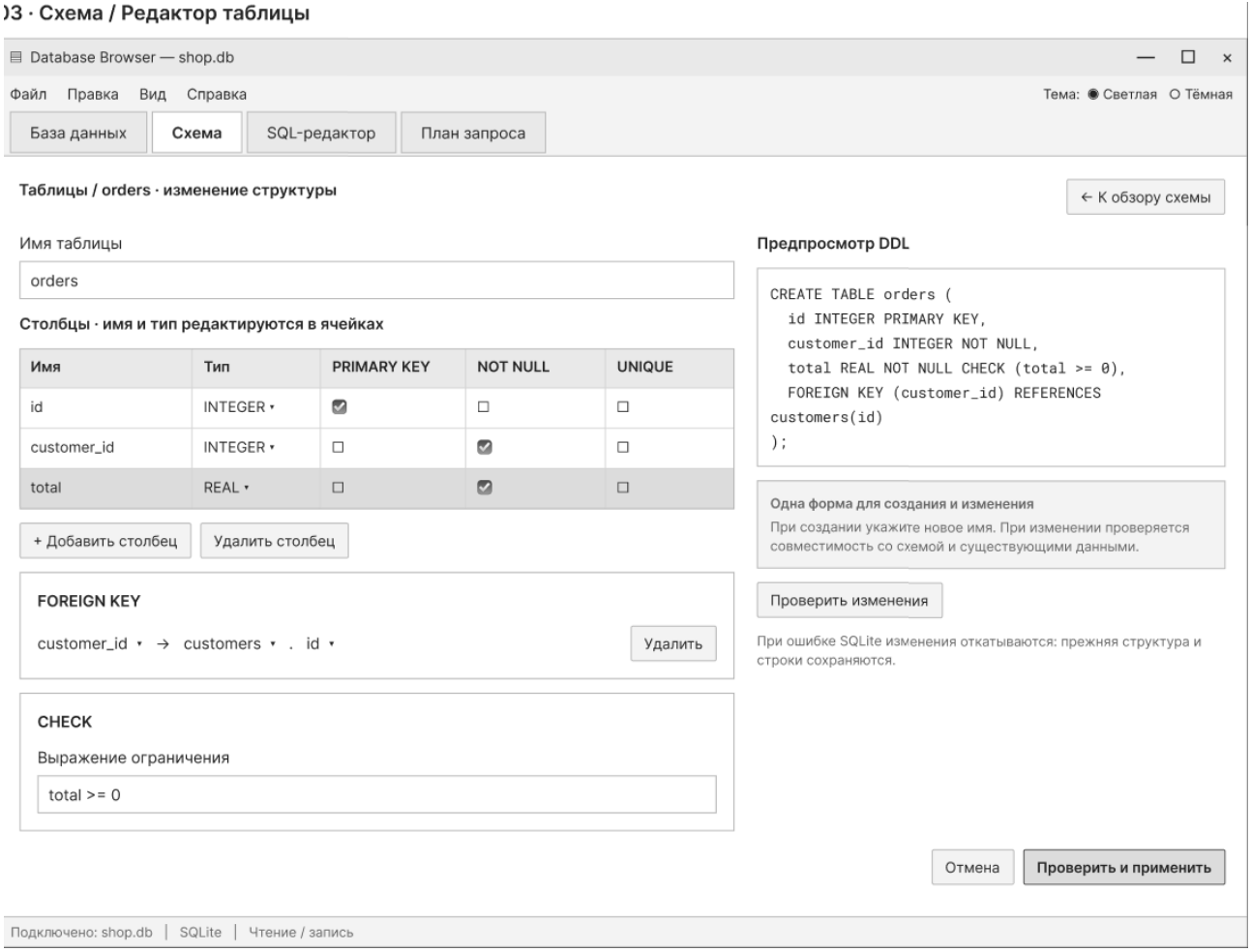
Sketches

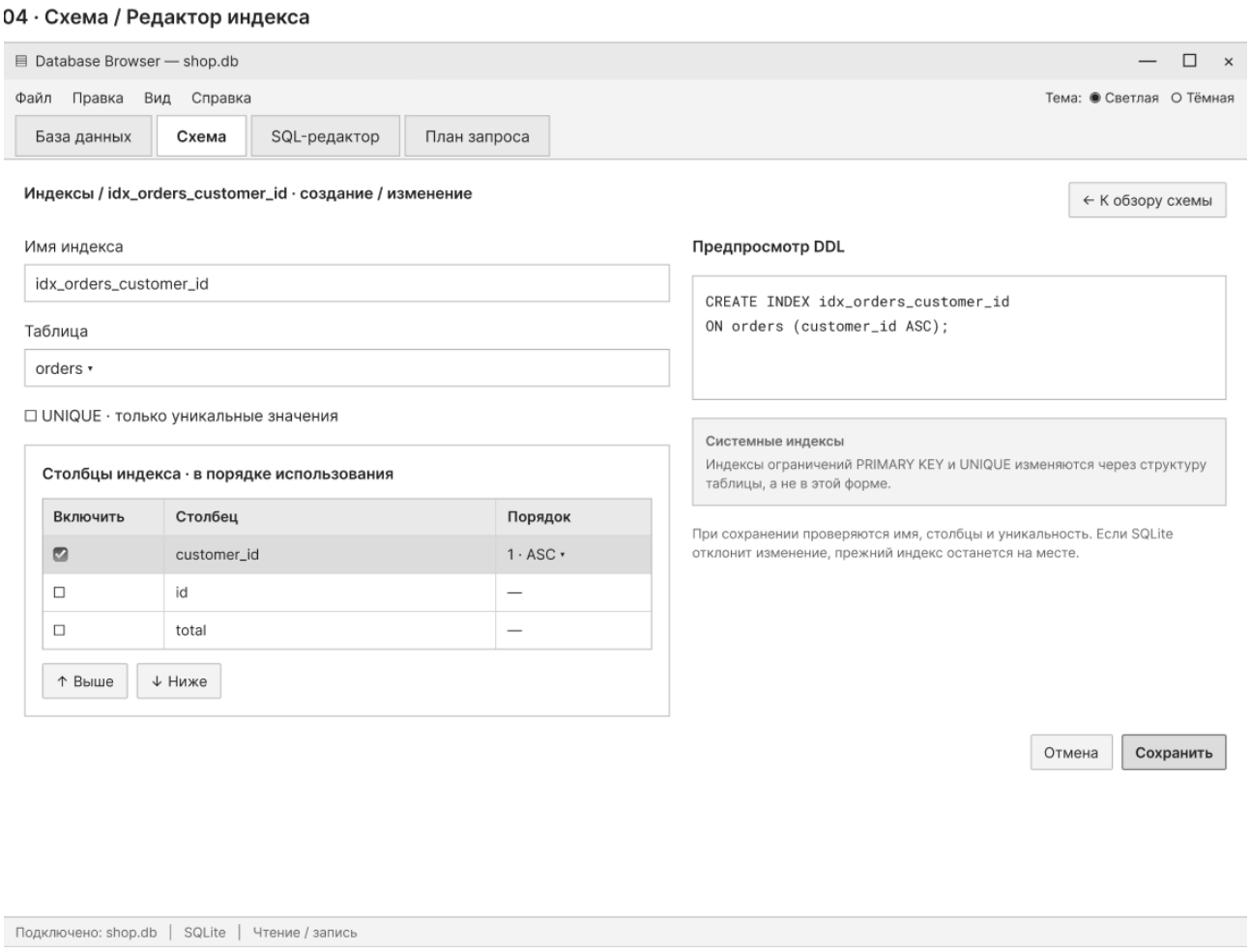
01 · База данных / Подключения



02 · Схема / Обзор




03 · Схема / Редактор таблицы
Database Browser — shop.db
Файл Правка Вид Справка
Тема: Светлая Тёмная
База данных
Схема
SQL-редактор
План запроса
Таблицы / orders · изменение структуры
← К обзору схемы
Имя таблицы
orders
Столбцы · имя и тип редактируются в ячейках
Имя
Тип
PRIMARY KEY
NOT NULL
UNIQUE
id
INTEGER
customer_id
INTEGER
total
REAL
+ Добавить столбец
Удалить столбец
FOREIGN KEY
customer_id → customers . id
Удалить
CHECK
Выражение ограничения
total >= 0
Предпросмотр DDL
CREATE TABLE orders (
id INTEGER PRIMARY KEY,
customer_id INTEGER NOT NULL,
total REAL NOT NULL CHECK (total >= 0),
FOREIGN KEY (customer_id) REFERENCES
customers(id)
);
Одна форма для создания и изменения
При создании укажите новое имя. При изменении проверяется совместимость со схемой и существующими данными.
Проверить изменения
При ошибке SQLite изменения откатываются: прежняя структура и строки сохраняются.
Отмена
Проверить и применить
Подключено: shop.db | SQLite | Чтение / запись


04 · Схема / Редактор индекса
Database Browser — shop.db
Файл Правка Вид Справка
Тема: Светлая Тёмная
База данных
Схема
SQL-редактор
План запроса
Индексы / idx_orders_customer_id · создание / изменение
← К обзору схемы
Имя индекса
idx_orders_customer_id
Таблица
orders
UNIQUE · только уникальные значения
Столбцы индекса · в порядке использования
Включить
Столбец
Порядок
customer_id
1 · ASC
id
total
↑ Выше
↓ Ниже
Предпросмотр DDL
CREATE INDEX idx_orders_customer_id
ON orders (customer_id ASC);
Системные индексы
Индексы ограничений PRIMARY KEY и UNIQUE изменяются через структуру таблицы, а не в этой форме.
При сохранении проверяются имя, столбцы и уникальность. Если SQLite отклонит изменение, прежний индекс останется на месте.
Отмена
Сохранить
Подключено: shop.db | SQLite | Чтение / запись

05 · Схема / Редактор представления

Database Browser — shop.db

Файл Правка Вид Справка

Тема: ◉ Светлая ○ Тёмная

База данных | **Схема** | SQL-редактор | План запроса

Представления / customer_orders · создание / изменение

← К обзору схемы

Имя представления

customer_orders

SELECT · редактируемый запрос

```
1 SELECT customers.name, orders.id AS order_id, orders.total
2 FROM customers
3 JOIN orders ON orders.customer_id = customers.id;
```

Предпросмотр определения

```
CREATE VIEW customer_orders AS
SELECT customers.name, orders.id AS order_id, orders.total
FROM customers
JOIN orders ON orders.customer_id = customers.id;
```

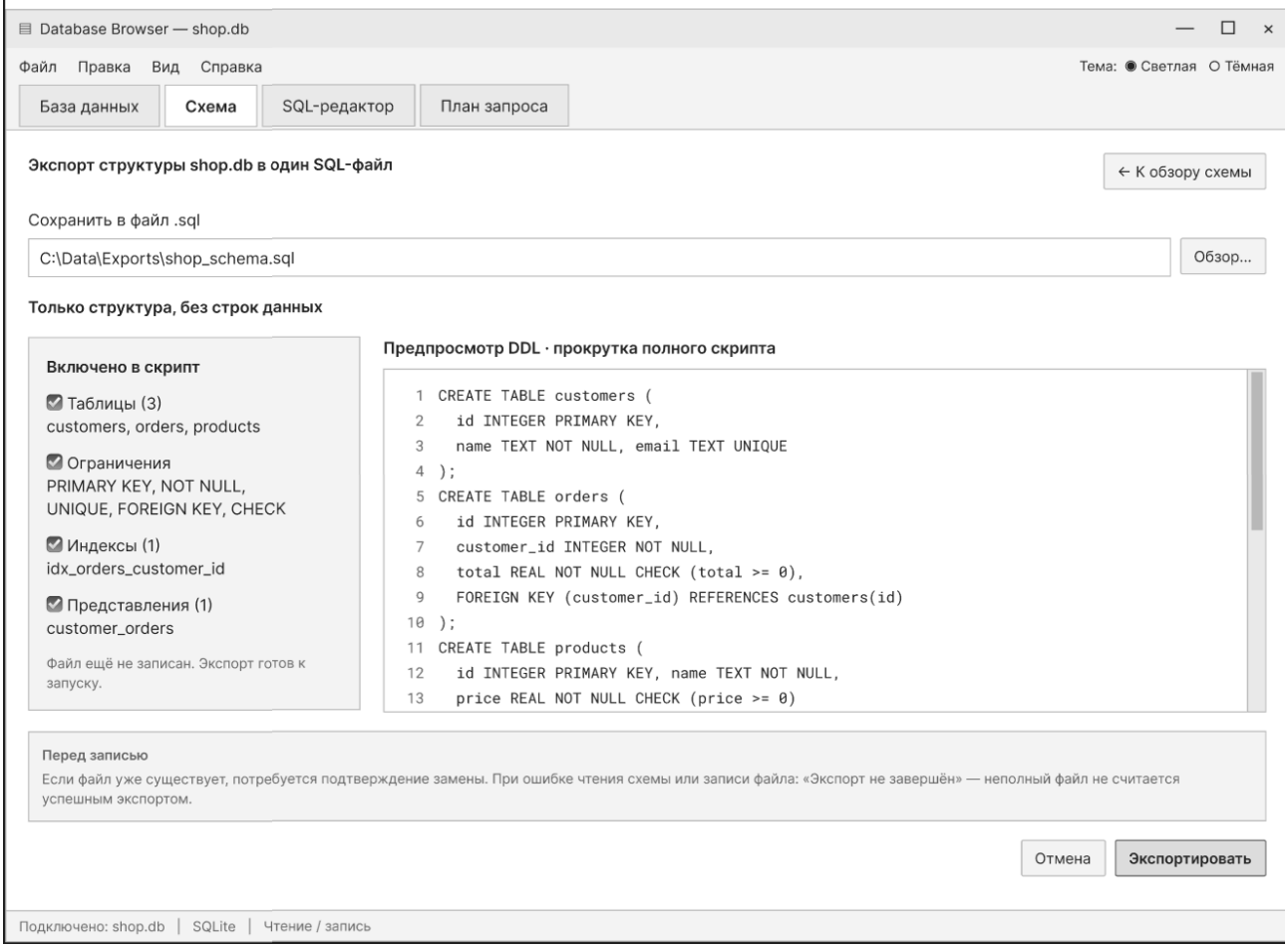
Область сообщений SQLite · сейчас ошибок нет

Пример при неверном поле: «no such column: orders.amount». Введённый SELECT остаётся в редакторе, прежнее представление сохраняется.

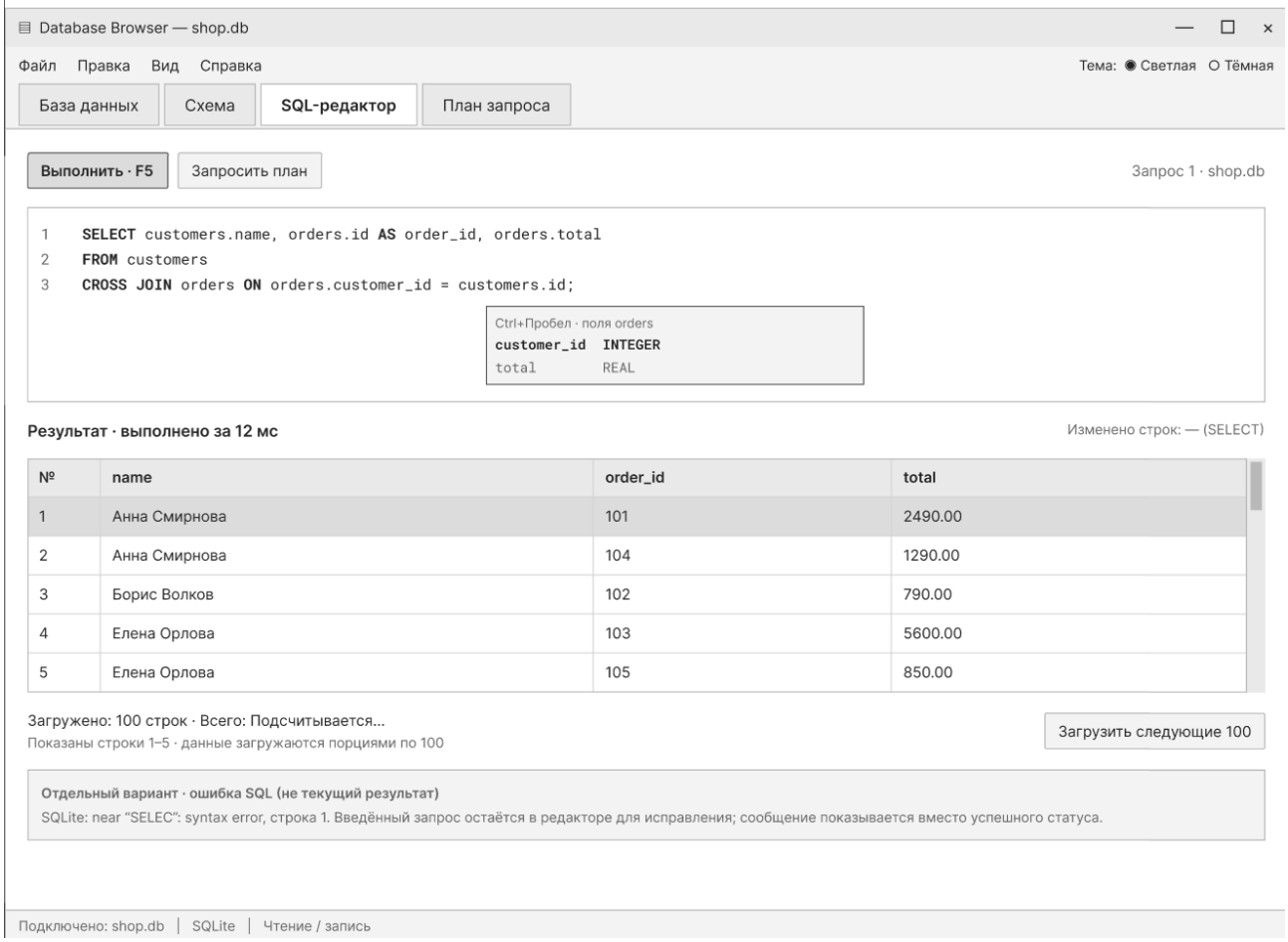
Отмена | **Сохранить**

Подключено: shop.db | SQLite | Чтение / запись

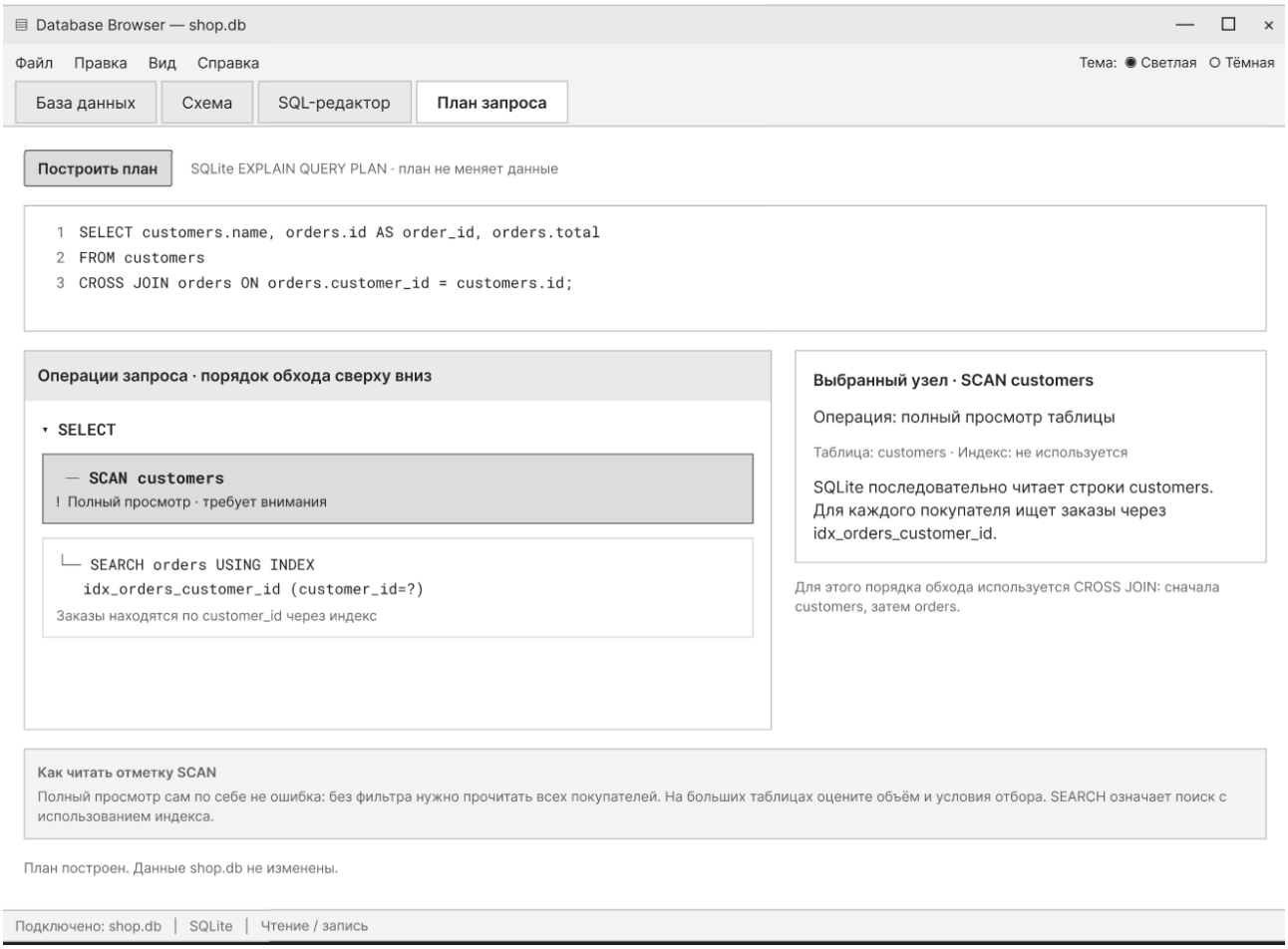
06 · Схема / Экспорт



07 · SQL-редактор / Результат



08 · План запроса



Precedents

Use cases for the first stage

This document describes the user's actions in the application for working with the relational database schema, executing SQL-queries, and analyzing their execution plans
In the first stage, SQLite is considered; support for PostgreSQL, MySQL, and collaboration relate to the possible further development of the project

Actors and boundaries

The main actor is the Database Browser user, who connects the database file and works with its contents and schema
Accounts, registration, and administrator roles are not included in the requirements of the first stage, so the same user type is specified for all scenarios.
Prerequisites, successful flow, unsuccessful and alternative flows, as well as a postcondition are given for each scenario separately

Functional areas and editor navigation

The editor is organized into four tabs. These tab names also define the feature groups for the use case diagram

Database tab — UC-01, UC-16, UC-17

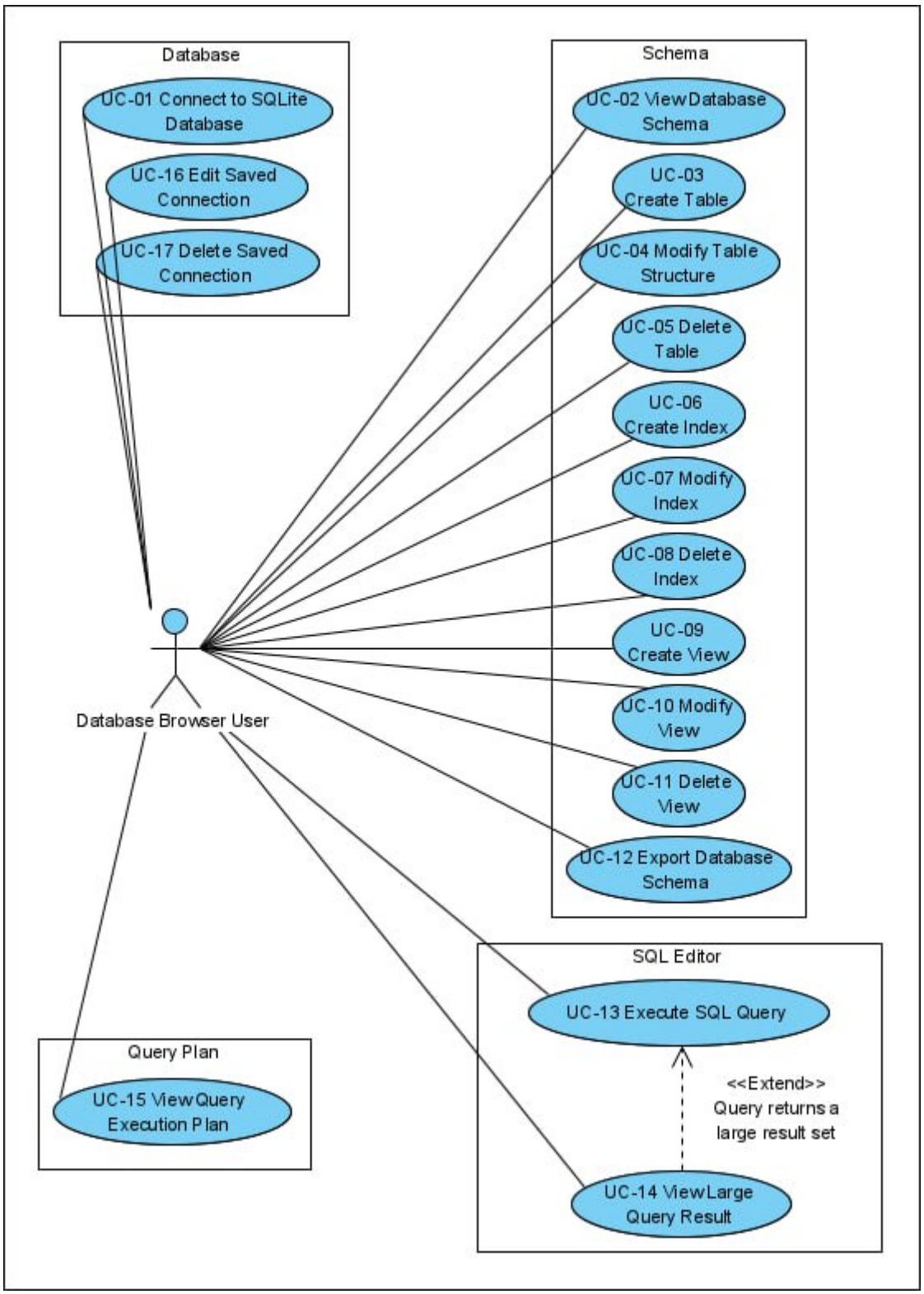
Schema tab — UC-02, UC-03, UC-04, UC-05, UC-06, UC-07, UC-08, UC-09, UC-10, UC-11, UC-12

SQL Editor tab — UC-13, UC-14

Query Plan tab — UC-15

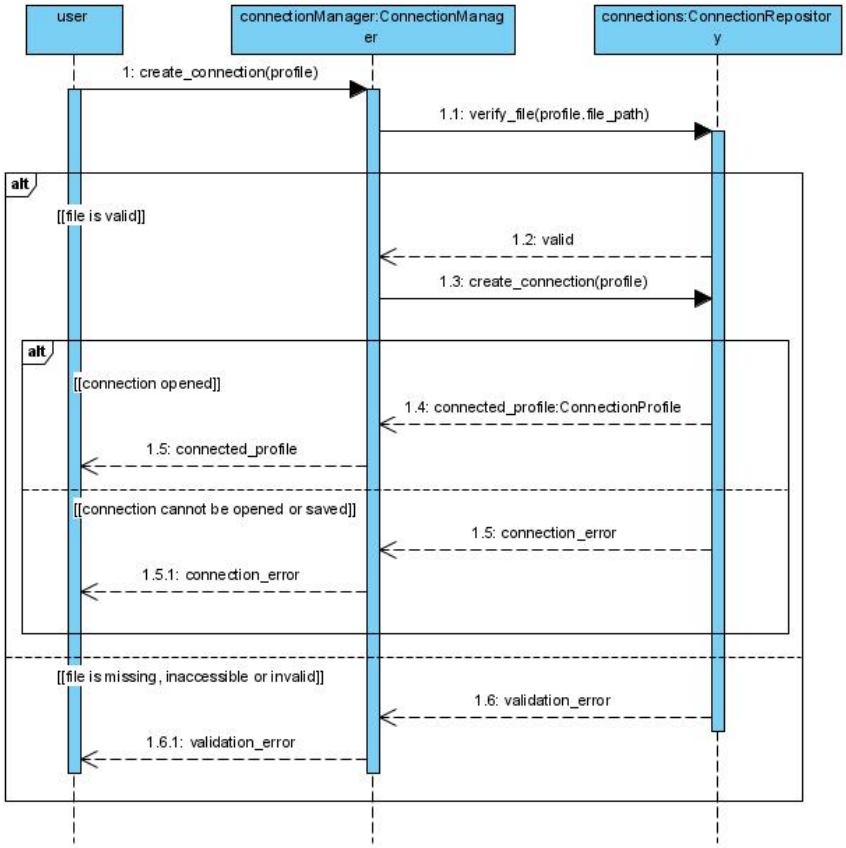
Inside the Schema tab, Tables, Indexes, Views, and Export are separate sections. Tab selection itself is a navigation step within a use case

The detailed use cases below retain their original identifiers. UC-14 extends UC-13 when the query returns a large result set


Database
UC-01 Connect to SQLite Database
UC-16 Edit Saved Connection
UC-17 Delete Saved Connection
Schema
UC-02 View Database Schema
UC-03 Create Table
UC-04 Modify Table Structure
UC-05 Delete Table
UC-06 Create Index
UC-07 Modify Index
UC-08 Delete Index
UC-09 Create View
UC-10 Modify View
UC-11 Delete View
UC-12 Export Database Schema
Database Browser User
SQL Editor
UC-13 Execute SQL Query
<<Extend>>
Query returns a large result set
UC-14 View Large Query Result
Query Plan
UC-15 View Query Execution Plan

Database tab
UC-01 Connect an SQLite database

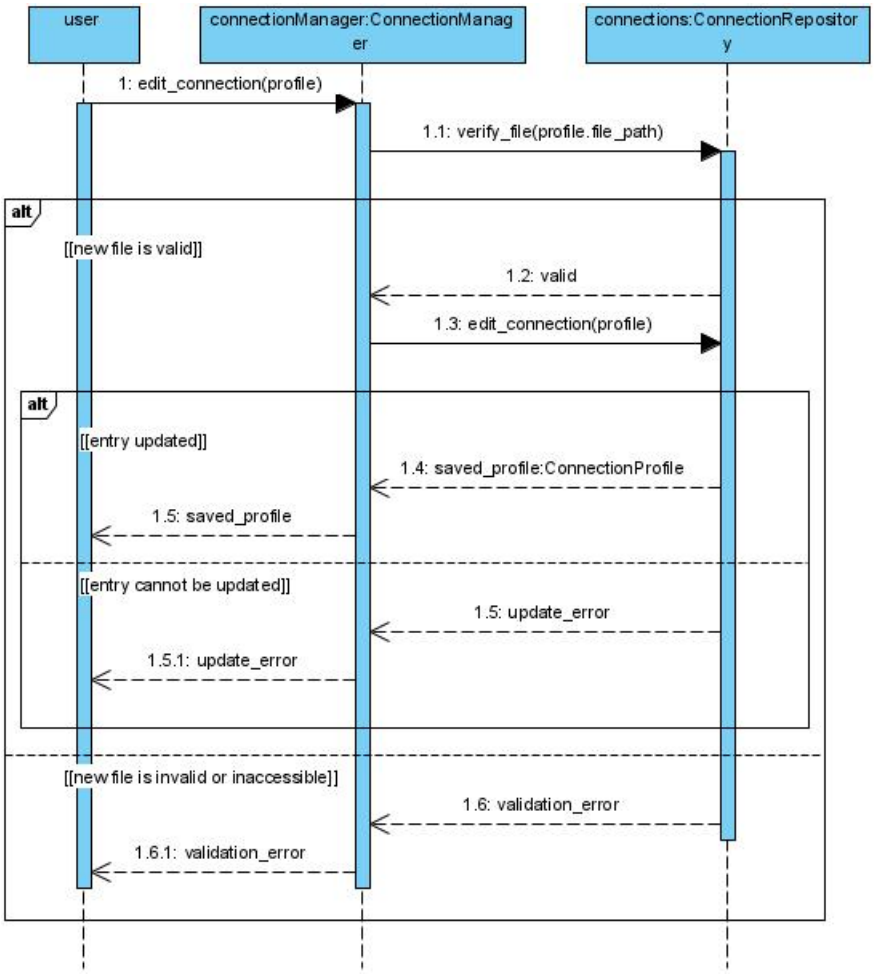
Actor	Database Browser User
Goal	To open the SQLite file and start working with the database
Prerequisites	The application is running
Successful flow	1 The user selects a saved connection or specifies an SQLite file for a new connection 2 The system checks the availability of the file, its format, and the integrity of the database 3 The system opens the connection and displays the database in the interface 4 If the file is specified for the first time, the system saves a record connections to re-open
Unsuccessful flow and alternatives	The user deselects the file-the connection is not being created The file is missing, unavailable, corrupted, or is not an SQLite database - the system reports the reason and does not open the connection
Postcondition on success	The connection is open, the database is available for viewing and executing queries



Saved connections

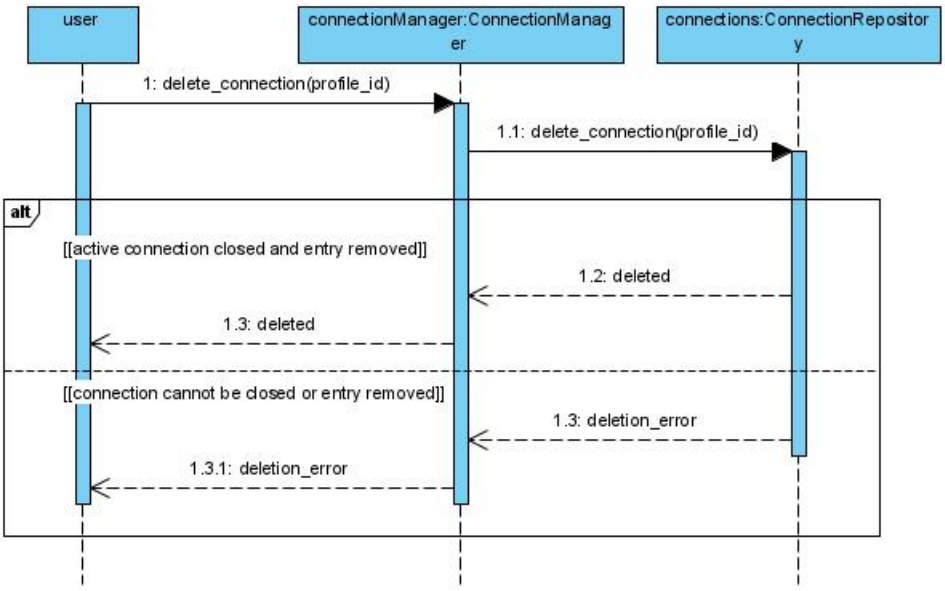
UC-16 Change a saved connection

Actor	Database Browser User
Goal	To update the parameters of the connection record to the SQLite file
Prerequisites	There is a saved connection record in the application
Successful flow	**1** The user selects the connection record for editing **2** The system shows the current path to the file **3** The user specifies a new path and saves changes **4** The System checks the availability, format, and integrity of the selected database and updates the connection record
Unsuccessful flow and alternatives	If the User cancels editing - the record does not change A new file is missing, unavailable, or incorrect - the system reports the reason, the previous parameters are saved
Postcondition on success	The connection record points to the verified SQLite file



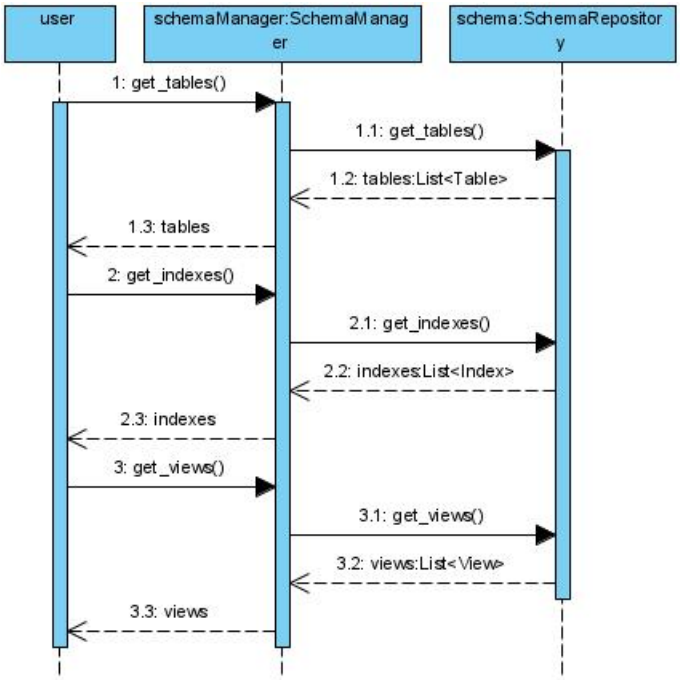
UC-17 Delete a saved connection

Actor	Database Browser User
Goal	To remove an unnecessary connection record from
Prerequisites	There is a saved connection record
Successful flow	**1** The user selects the connection record to delete **2** The system explains that the database file will not be deleted and requests confirmation **3** The user confirms the deletion **4** The System closes the open connection and deletes its entry from the list
Unsuccessful flow and alternatives	When the user cancels the deletion - the record and the open connection are saved The connection failed to close or the record failed to delete - the system reports the reason and shows the actual state
Postcondition on success	The connection record is deleted from the application, the SQLite file is saved to disk



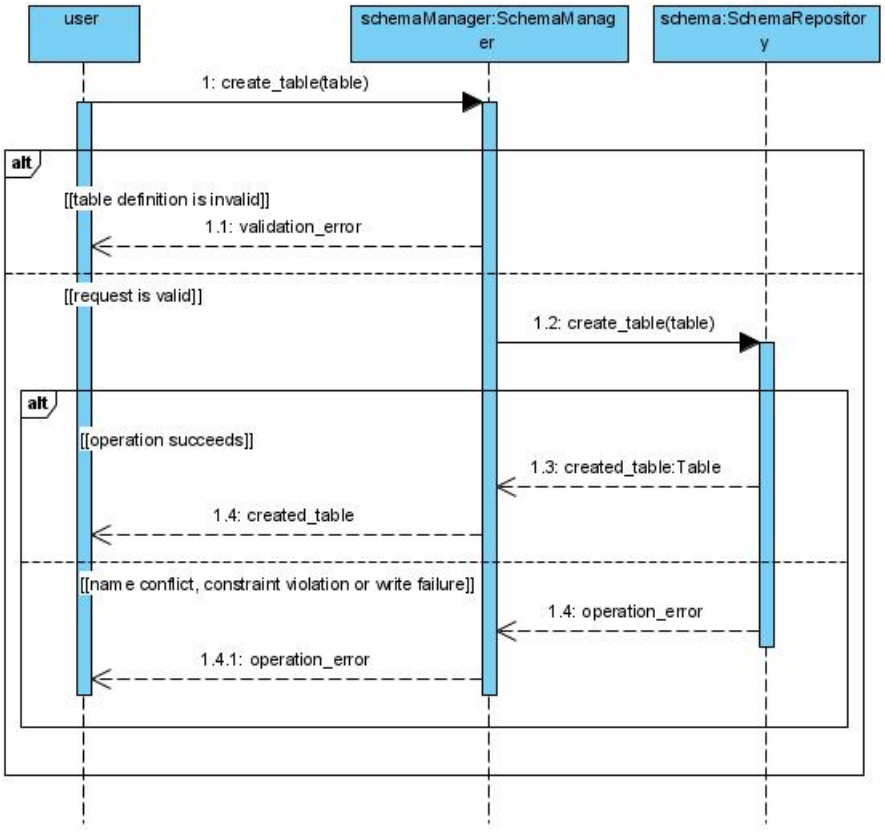
Schema tab
UC-02 View Database Schema

Actor	Database Browser User
Goal	To find out the composition and structure of open database objects
Prerequisites	SQLite database is connected
Successful flow	**1** User selects the connected database **2** System shows tables, indexes, and views **3** User selects a schema object **4** System shows its definition: for a table-columns, types, and constraints; for an index-a table and indexed columns; for the view - defining request
Failed flow and alternatives	There are no objects in the database - the system shows an empty list Object information could not be read - the system reports an error and saves the connection
Postcondition on success	The User sees the structure of the selected object, the database is not changed



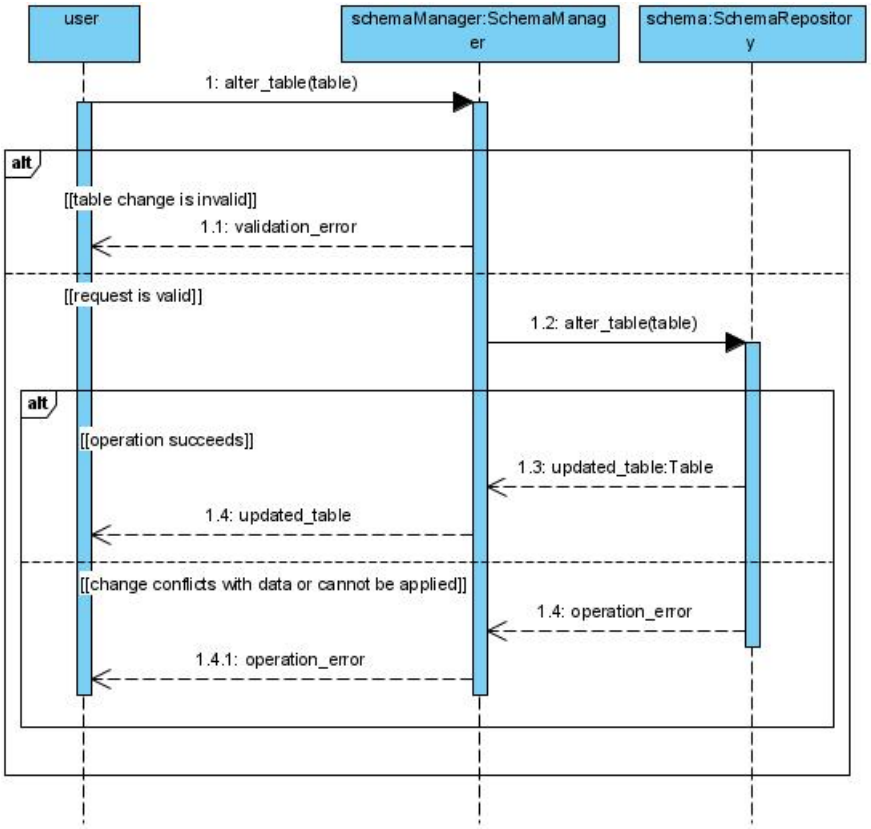
Managing the database schema
UC-03 Create a Table

Actor	Database Browser User
Goal	To add a table with specified columns and constraints
Prerequisites	SQLite database is connected and writable
Successful Flow	**1** User sets the table name, columns, and data types **2** If necessary, the user sets PRIMARY KEY, FOREIGN KEY, UNIQUE, CHECK, and NOT NULL **3** User confirms creation **4** The system passes the SQLite definition and updates the displayed schema
Unsuccessful flow and alternatives	The user cancels creation - the schema does not change The name is occupied, the definition is incorrect, or the entry is forbidden - the system reports an error, the table is not created
Postcondition on success	The table is created and displayed in the schema



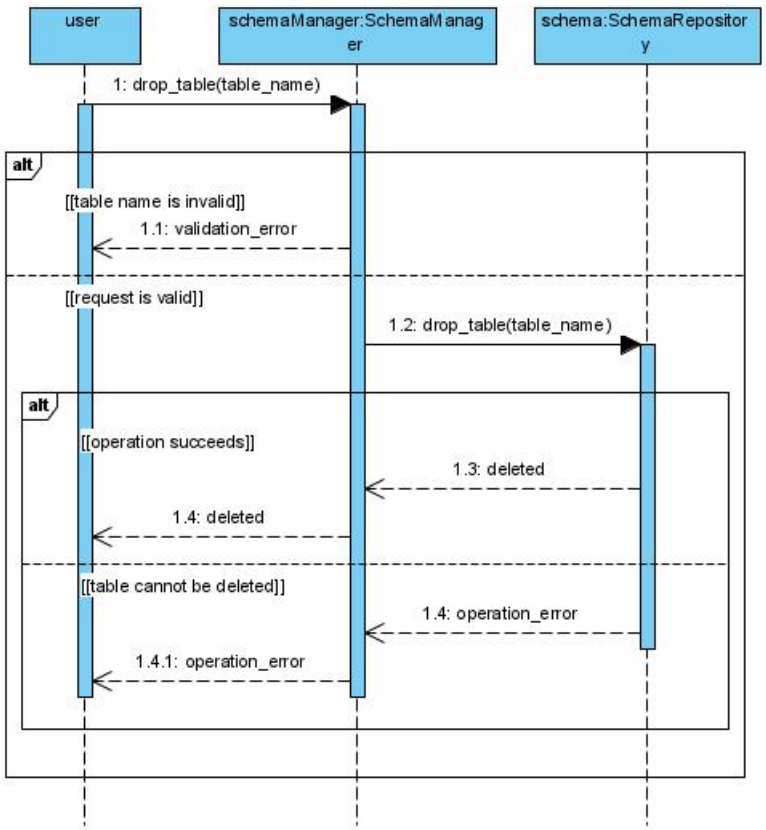
UC-04 Change Table Structure

Actor	Database Browser User
Goal	To change columns or constraints of an existing table
Prerequisites	SQLite database is connected and writable, the table exists
Successful flow	**1** User selects the table and sets the required change in its structure **2** System checks the change's applicability to the current schema and data **3** User confirms the change **4** System applies the entire change and updates the displayed schema
Unsuccessful flow and alternatives	If the user cancels the change - the table is saved in the same form The change is not possible using SQLite, violates restrictions or conflicts with data - the system reports the reason and retains the same structure An error occurred during application - the system rolls back the change and shows the actual state of the schema
Postcondition on success	The table structure corresponds to the saved definition



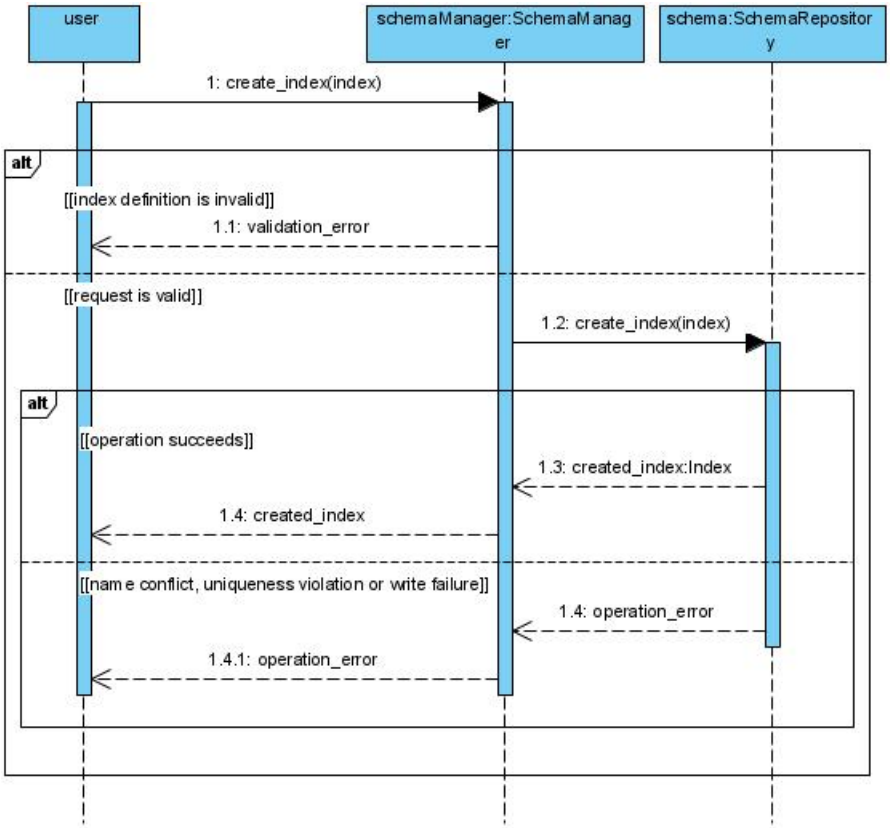
UC-05 Delete Table

Actor	User Database Browser
Goal	Delete the table along with its data
Prerequisites	SQLite database is connected and writable, the table exists
Successful flow	**1** User selects the table to delete **2** The system warns about deleting the table and its rows and requests confirmation **3** The user confirms the deletion **4** The system deletes the table and updates the schema
Unsuccessful flow and alternatives	The User refuses from deletion - the table and data are saved SQLite cannot delete the table - the system reports the reason and shows the actual state of the database
Postcondition on success	The table and its data are deleted from the database



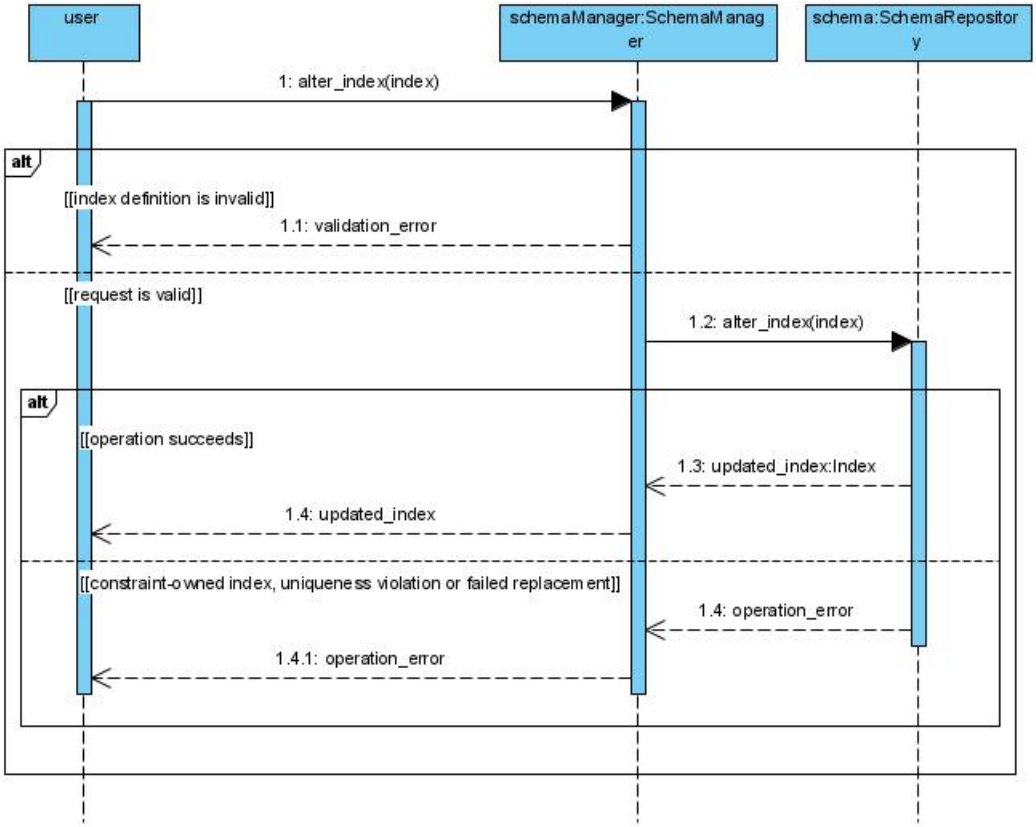
UC-06 Create an Index

Actor	Database Browser User
Goal	To create an index for selected table columns
Prerequisites	SQLite database is connected and writable, the table exists
Successful flow	**1** User sets the index name and one or more table columns **2** If necessary, the user specifies the uniqueness attribute **3** User confirms creation **4** The system creates an index and updates the schema
Unsuccessful flow and alternatives	User cancels creation-the schema does not change The name is occupied, the column does not exist, or the available data violates the uniqueness requirement - the system reports an error, the index is not created
Postcondition on success	The Index is created and displayed in the schema



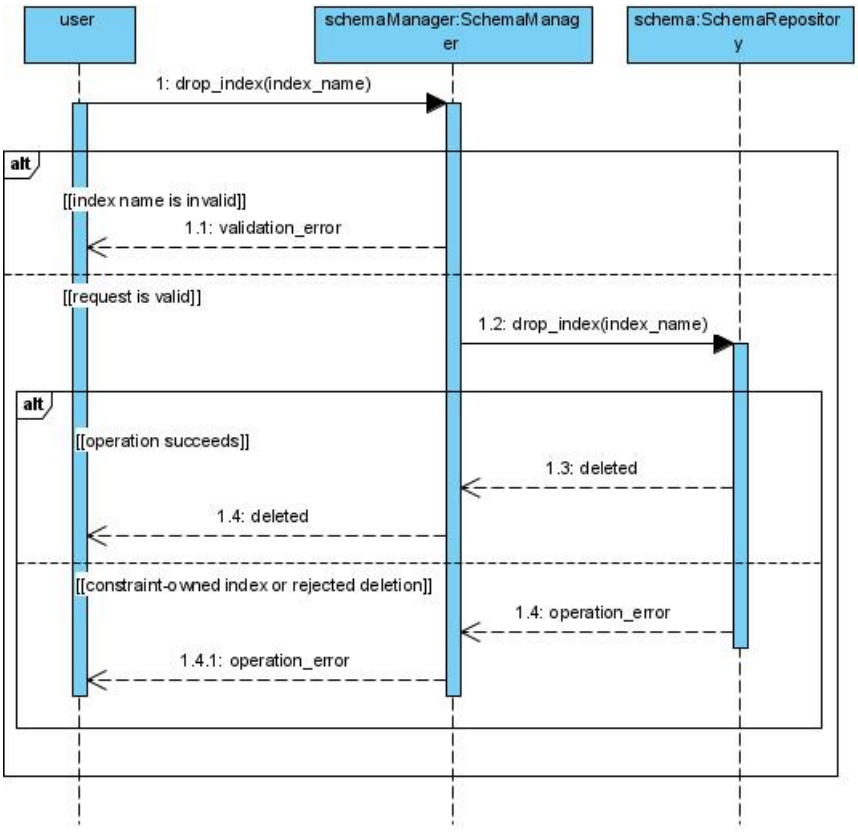
UC-07 Change the Index

Actor	User Database Browser
Goal	To change the definition of the user-created index
Prerequisites	SQLite database is connected and writable, the user index exists
Successful flow	**1** The user selects the index and sets a new set of columns or a unique attribute **2** The system checks the new definition **3** The user confirms the change **4** The system replaces the index in one operation and updates the schema
Unsuccessful flow and alternatives	The user cancels the change - the previous index is saved The definition is incorrect or the data violates the uniqueness requirement - the system reports an error and saves the previous index The index created by the PRIMARY KEY or UNIQUE constraint is selected - the system suggests changing the table structure via the UC-04
Postcondition on success	The User index has a new definition


user
schemaManager:SchemaManag
er
schema:SchemaRepositor
y
1: alter_index(index)
alt
[[index definition is invalid]]
1.1: validation_error
[[request is valid]]
1.2: alter_index(index)
alt
[[operation succeeds]]
1.3: updated_index:Index
1.4: updated_index
[[constraint-owned index, uniqueness violation or failed replacement]]
1.4: operation_error
1.4.1: operation_error

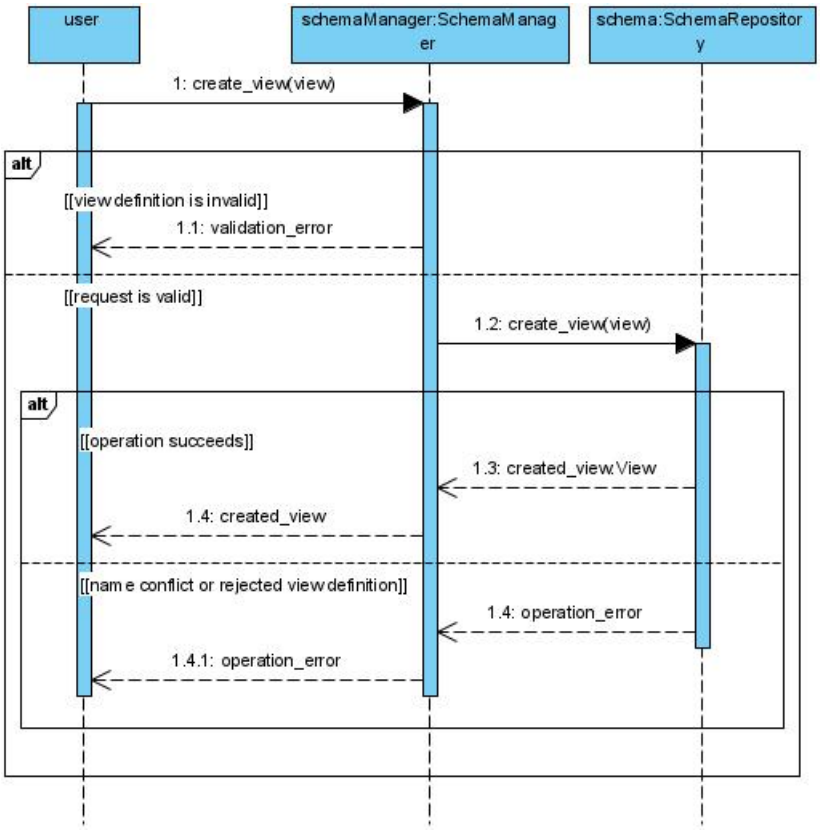
UC-08 Delete Index

Actor	User Database Browser
Goal	Delete user-created index
Prerequisites	SQLite database is connected and writable, user index exists
Successful flow	**1** User selects index **2** System shows linked table and asks for confirmation **3** User confirms deletion **4** System deletes index and updates schema
Unsuccessful flow and alternatives	User cancels action-index is saved Index cannot be deleted separate from the table constraint, or SQLite rejects the operation - the system reports the reason and saves the actual state of the schema
Postcondition on success	The index is deleted, the table rows are saved



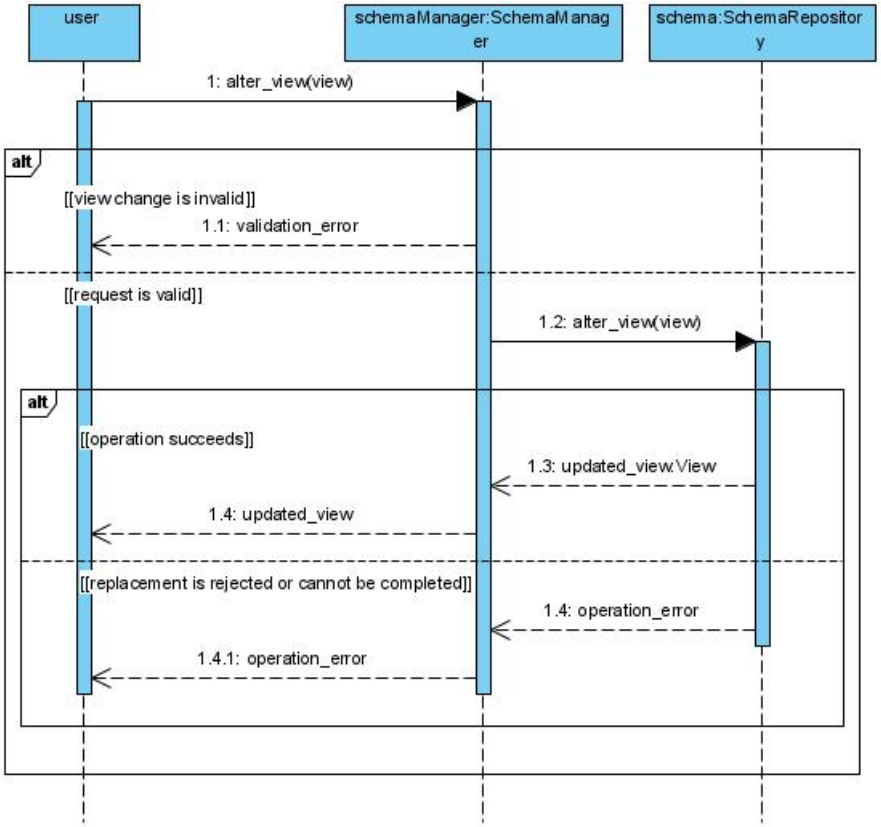
UC-09 Create a View

Actor	Database Browser User
Goal	To add a named view based on SELECT to the schema
Prerequisites	SQLite database is connected and writable
Successful flow	**1** User sets the view name and defining SELECTquery **2** User confirms creation **3** System creates the view and updates the schema
Unsuccessful flow and alternatives	User cancels creation-schema does not change Name is occupied or SQLite is used rejects the definition - the system shows an error, the view is not created
Postcondition on success	The view is created and displayed in the schema



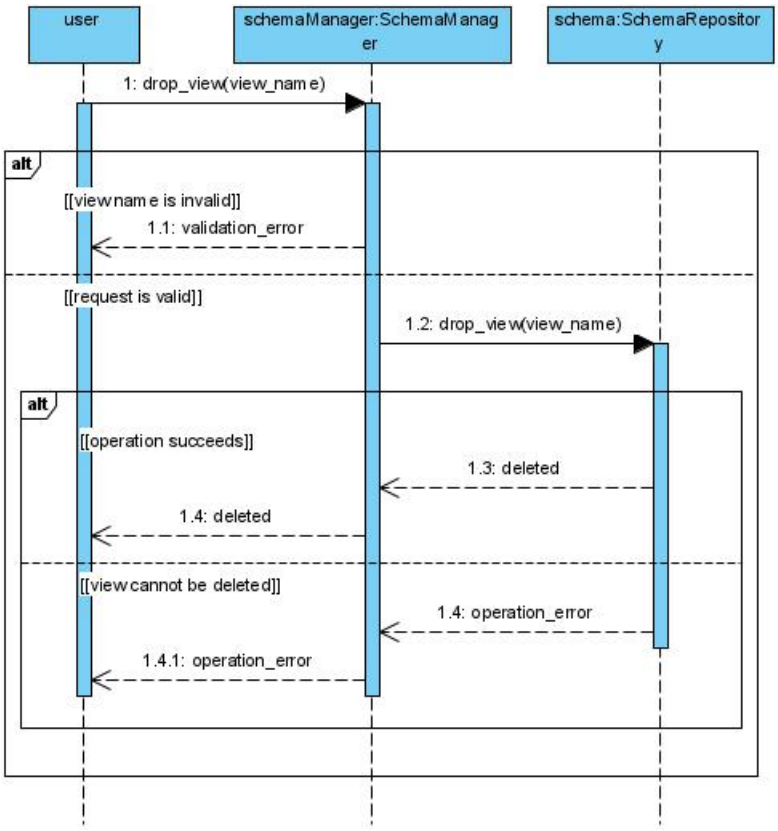
UC-10 Change the view

Actor	Database Browser User
Goal	To change the name or defining query of the view
Prerequisites	SQLite database is connected and writable, the view exists
Successful flow	**1** User selects the view and views its current definition **2** User sets a new name or SELECTquery and confirms the change **3** System replaces the definition as a single operation and updates the schema
Unsuccessful flow and alternatives	The user cancels editing - the view does not change SQLite rejects the new definition or the replacement does not complete - the system reports an error and saves the previous view
Postcondition on success	Next subsequent accesses to the view use the saved definition



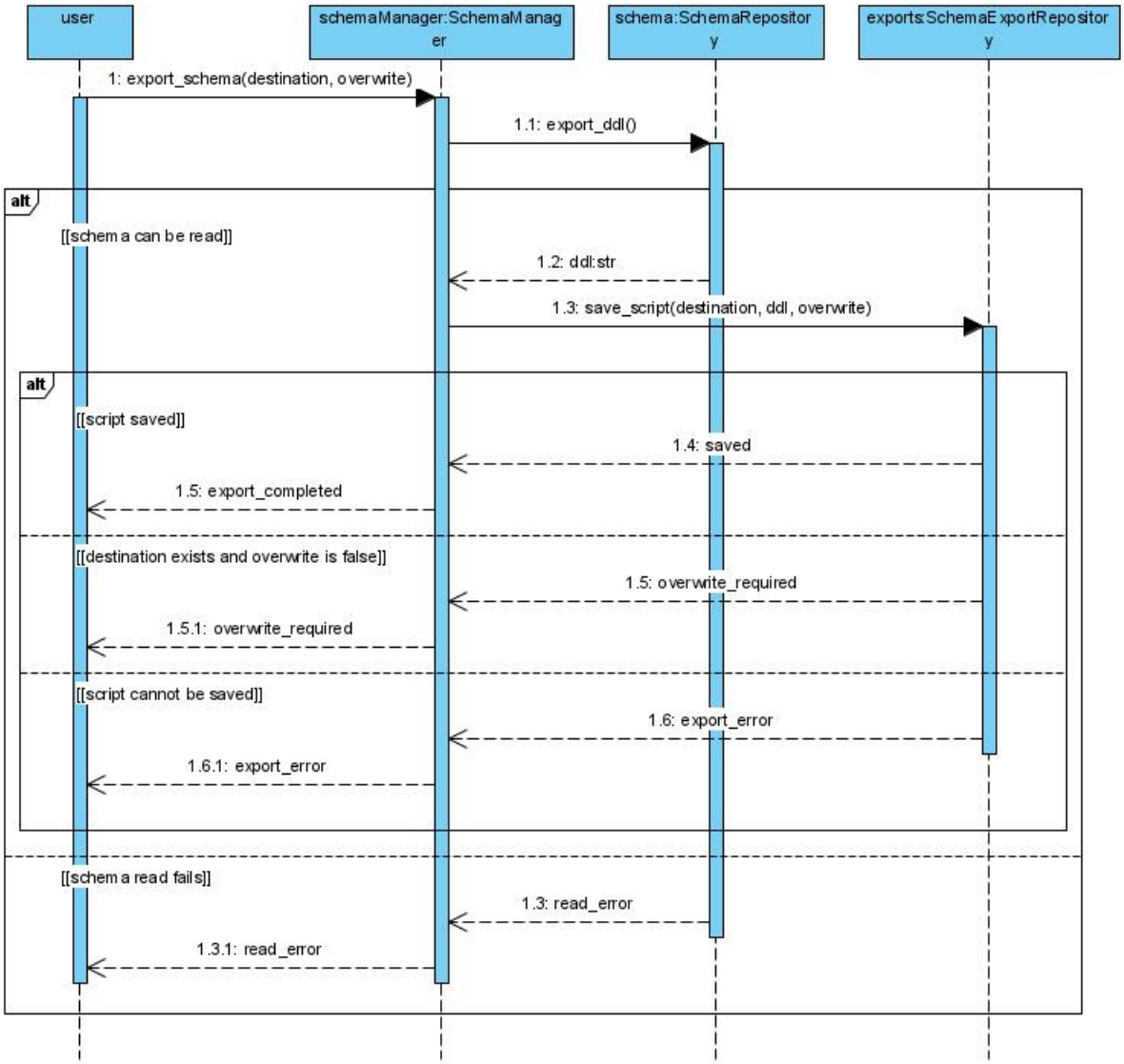
UC-11 Delete a View

Actor	User Database Browser
Goal	To remove a view from the schema
Prerequisites	SQLite database is connected and writable, the view exists
Successful flow	**1** User selects the view to delete **2** System requests confirmation **3** User confirms deletion **4** System deletes the view and updates the schema
Unsuccessful flow and alternatives	User cancels the action - the view is saved SQLite rejects deletion - system reports error and shows the actual state of the schema
Postcondition on success	The view is deleted, data from the source tables is saved



UC-12 Export Database Schema

Actor	Database Browser User
Goal	To get a single SQLscript with the definition of the current schema
Prerequisites	SQLite database is connected, the schema is readable
Successful flow	**1** The user selects the schema export and specifies the path to the SQLfile **2** The system collects DDL tables with constraints, indexes, and views **3** The system saves a single script and reports successful export
Unsuccessful flow and alternatives	The user cancels saving - no new file is created The file already exists - the system asks for confirmation of replacement The schema could not be read or the file could not be written - the system reports the reason, incomplete export is not issued as completed
Postcondition on success	The SQL-file contains definitions of schema objects without data strings

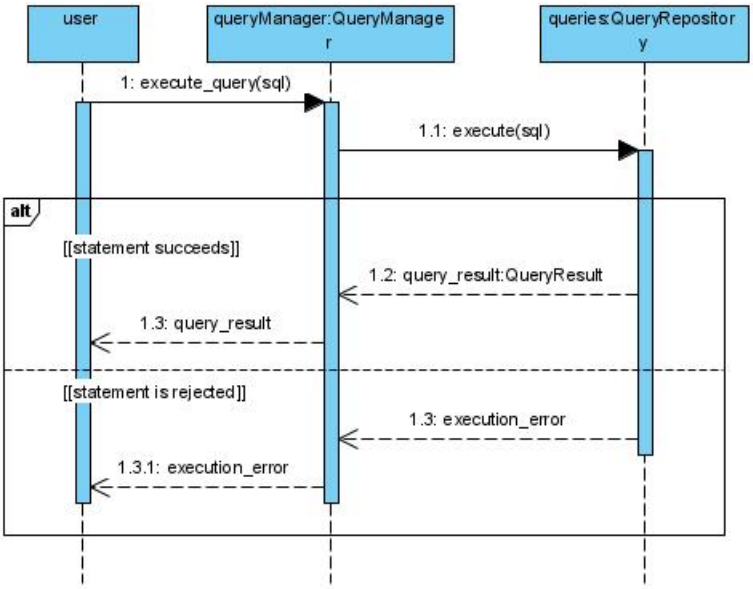


SQL Editor tab

Queries and analysis

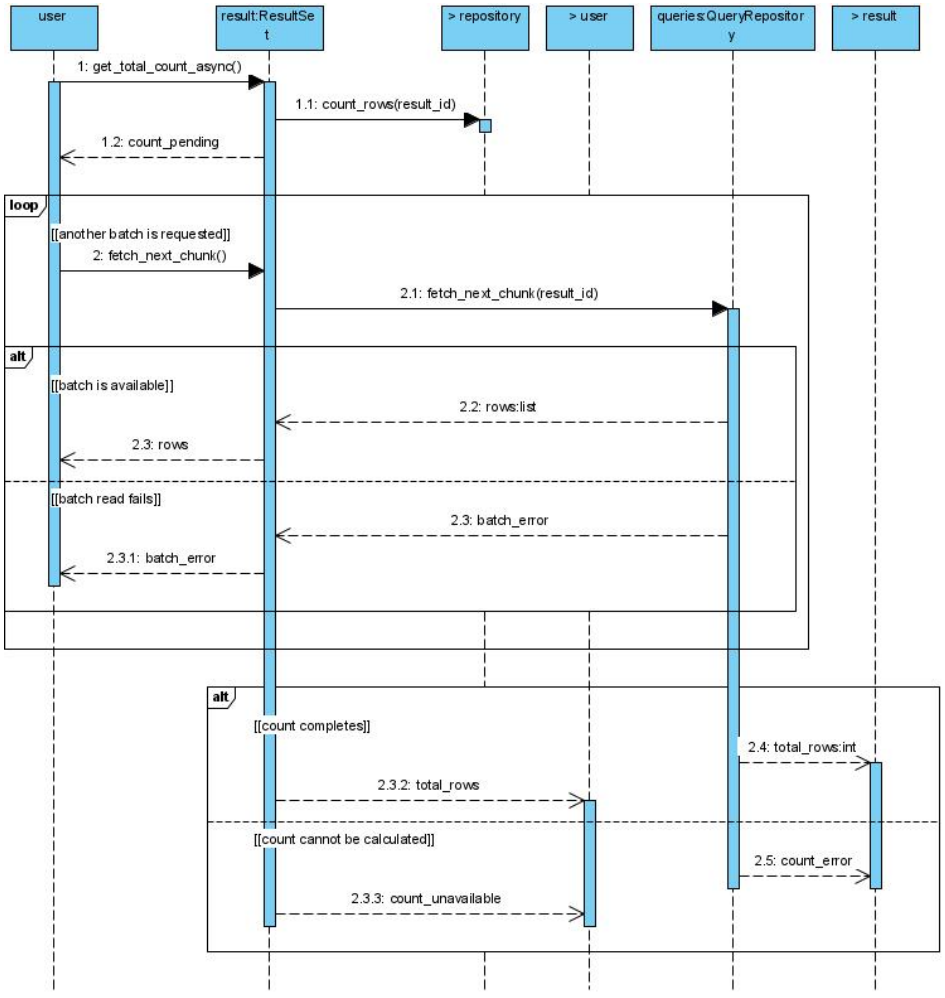
UC-13 Execute an SQLQuery

Actor	Database Browser User
Goal	To Execute SQL and get rows or result of changing data or schema
Prerequisites	SQLite database is connected
Successful flow	**1** User enters an SQLquery and runs it **2** The system executes the query without blocking the interface **3** For a query with returned rows, the system shows a table result, for changing data or schema - the result of execution **4** The System shows available metrics: execution time and the number of affected rows, if applicable; after changing the schema, updates its display
Unsuccessful flow and alternatives	The User cancels input before starting-the database does not change SQL contains an error, accesses a missing object, or SQLite rejects the operation-the system shows the reason and remains functional The query did not return any rows - the system shows empty result without an error message
Postcondition on success	The request is completed, the user sees its result and available statistics



UC-14 View Large Query Result

Actor	Database Browser User
Goal	To examine parts of a large SELECT result and estimate its size
Prerequisites	Executed a UC-13 query returning a large set of rows
Successful flow	**1** The system shows the first batch of rows **2** The user requests the next batch or scrolls through the result **3** The system loads the necessary rows without loading the entire result into memory **4** The System calculates the total number of rows separately and shows it after the calculation is completed along with the available query statistics
Unsuccessful flow and alternatives	The user stops viewing - the displayed rows remain available Upload the next batch failed - the system reports an error and saves the already shown rows Counting the total number of rows failed - the system reports that the indicator is not available, without hiding the result itself
Postcondition on success	The User viewed required portions of the result and got the available statistics



Query Plan tab

UC-15 View Request Execution Plan

Actor	Database Browser User
Purpose	To understand the order of operations and use of indexes when executing SQL
Prerequisites	SQLite database is connected, the user entered a query for analysis
Successful flow	**1** The user requests a plan for the entered SQL **2** The system receives the SQLite plan **3** The system shows the steps of the plan in the form of a tree or graph **4** The System marks operations for verification, such as a full table scan
Unsuccessful flow and alternatives	The request is incorrect or the plan was not received - the system shows the reason and saves the entered SQL
Postcondition if successful	The User sees the plan and can evaluate the selectedSQLite operations without changing the database data

